

The *NIX-hacker's Guide to Systemd

It's taking the Linux world by storm. Fedora already uses it, OpenSUSE is switching over to it, and it achieves a sub-two-second start-up time on SSDs. In this article, I explain how to tame the beast that Systemd is.

After a very successful class in Vector Algebra, I came back home to see Kolkata Knight Riders restrict Mumbai Indians to 108 in 19.1 overs, going on to win the match. Feeling on top of the world (which can be a liability, as we'll see), I decided to replace SysV-init on Arch with Systemd.

But first of all, here's a little background on Systemd. The UNIX process model is hierarchical. Every process must have a parent process. When you open up your terminal and run the package manager, it is a child process of your shell, which is a child process of the terminal emulator, which is a child process of, say, Gnome Shell, which is a child process of *gnome-session*, which is a child process of X11, and so on and so forth. Keep traversing this, and you'll end up at *init*, which is the parent of all processes. *Init* is special; its parent is the Linux (or any UNIX for that matter) kernel. It has a special PID, 0. If you kill *init*, your entire system dies.

Init needn't actually be called *init*, though. Any program with PID 0 has the same privileges as *init* does. You can start any program as PID 0—even *bash*. All you need to do is pass an *init=/bin/bash* parameter to the kernel when booting up, and a Bash shell will greet you (this is a classic way to defeat almost all security on a UNIX system and reset the root password, which is why most boot-loaders support password protection).

On Darwin and Mac OS X, *init* is provided by Launchd; and on Ubuntu, by Upstart. On most Linux distributions, Solaris and BSD, *init* is provided by SysV-init, which, as the name suggests, comes from System V UNIX. Those at Fedora, the biggest out-of-laboratory computer science experiment ever to be carried out, thought about pulling SysV-init for a next-generation replacement, calling it Systemd.

Interestingly, the folks at Fedora had ambitious plans for replacing SysV-init with Upstart. They used Upstart in Fedora 14, until a bloke called Lennart Poettering, who works for

Red Hat, came up with Systemd, which doesn't really stand for anything, but the name does suggest that it's the 'System Daemon'. It takes a bunch of ideas out of Mac's Launchd, and it successfully slashed my boot times by 10 whole seconds (to 18.9 seconds, 16.5 seconds of which is spent in disk I/O on a 5400 RPM SATA 3GB/s drive). So what makes it so damn effective? I have two words for you—massive parallelisation.

Let's go back to 1983, when System V UNIX was the hottest piece of computer software around. AT&T designed its *init* system to be slightly different from that of the BSD style. System V used shell scripts, directories and a bunch of symlinks. Every runnable program had a script, which would accept a defined set of arguments—*start*, *stop*, *restart* and *status*. There were 'runlevels', or numbers that configured the system to a specific state. Each runlevel was given a directory, and the service scripts were symlinked into these directories. The symlinks were given serial names. Then, when the system started, *init* would run through the runlevel directory, execute the service scripts with the *start* argument in alphabetical order, and drop you into a TTY when done. When it was time to shut down the system or switch runlevels, it would first shut down services in the current runlevel by running the service scripts with the *stop* argument in reverse alphabetical order, then halt or enter a new runlevel.

In today's multi-core asynchronous I/O-capable computers, this is grossly inefficient. Why do all the services need to be started or stopped serially?

Systemd solves this problem. Systemd's ideas are hardly new, but the implementation is top-notch. Think about Upstart, HAL, ConsoleKit and Xined all rolled into one. That's Systemd for you.

Systemd hates shell scripts. Services in Systemd are called units, and these units are simple INI files that someone without any programming knowledge can write in minutes. They describe the executable to run, the units that this